

Kybeurre demi-sel

Gist

En prenant un Q trop grand, on singularise la clef dans l'espace des chiffrés : la clef chiffrée devient un vecteur relativement petit dans la lattice cible ; on peut la récupérer par des algorithmes de recherche de plus petit vecteur.

Morale : Contrairement à RSA, faire plus gros est parfois contre-productif ; le secret sort comme un arbre au milieu du désert.

Solution

Idée générale

En lisant le code du sniffer python, on note

1. Qu'il broadcast le FLAG chiffré à intervalles réguliers (healthcheck) ;
2. Que c'est aussi le secret de la clef publique Kyber ;
3. Qu'elle change de temps en temps.

On doit donc casser la clef publique Kyber à partir d'un grand nombre d'échantillons. On note, par ailleurs, que Q est très grand par rapport à une implémentation normale de LWE. En temps normal, on aurait $\log_2(Q) \sim n$. Ici, c'est le double.

En LWE, la génération d'une clef publique se synthétise par l'équation

$$b = A \cdot s + e \bmod Q$$

où (A, b) est la clef publique, Q le module public connu, s est le secret et e n'est jamais divulgué. On note enfin n la dimension.

La crypto en lattices repose sur le fait que, en grande dimension, trouver un vecteur proche d'un point d'un réseau – ou le plus petit vecteur –, est un problème difficile. LWE ajoute le terme d'erreur e précisément pour produire un point qui n'est pas dans le réseau (lattice) engendré par A mais suffisamment proche.

Or ici, avec un Q aussi grand, les cellules de la lattice sont énormes. Malgré l'erreur, le point généré n'est jamais suffisamment loin du réseau pour que le retrouver ne devienne difficile : les heuristiques comme LLL vont marcher.

Ne reste qu'à construire la matrice d'attaque et laisser sage résoudre le problème.

Réduction à LLL

On va construire un réseau dont le vecteur le plus court est le secret de chiffrement (secret_vector dans le code python).

On sait que

$$b = A \cdot s + e \bmod Q$$

$$\exists k \mid b = A \cdot s + e + kQ$$

$$\exists k \mid A \cdot s + kQ - b = e$$

Et ce, pour chacun des broadcasts partageant un même secret. Ici, d'après le code python, e va être un vecteur court – plus court que tout autre vecteur de l'équation. On va donc créer une matrice telle que e soit un point du réseau.

Soit m le nombre de samples à notre disposition, I_m la matrice identité de dimension m . Soit

$$\mathcal{A} = \begin{pmatrix} A_1 \\ A_2 \\ A_3 \\ \vdots \end{pmatrix}$$

$$\mathcal{C} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \end{pmatrix}$$

$$e = \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ \vdots \end{pmatrix}$$

les matrices empilant les matrices lignes des clefs publiques des différents samples.

$\mathbb{A}$ est une matrice à m lignes et n colonnes. $\mathbb{b}$ à m lignes et une colonne.

On pose la matrice de réseau

$$\Lambda = \begin{pmatrix} Q \cdot I_m & 0 & 0 \\ \mathcal{A}^T & I_n & 0 \\ -\mathcal{C}^T & 0 & 1 \end{pmatrix}$$

$(e_1, \dots, e_n, s_1, \dots, s_n, 1)$ (abrégé $(e, s, 1)$) en est un (petit) vecteur. En effet, on constate que le vecteur à coefficients entiers $v = (k_1, \dots, k_n, s_1, \dots, s_n, 1)$ engendre bien $(e, s, 1)$:

$$v\Lambda = (e, s, 1)$$

.

$(e, s, 1)$ est par ailleurs un petit vecteur. Il y a de grandes chances que LLL nous le renvoie.

Le reste est un détail d'implémentation sage, et d'appel à `M.LLL()`.